7 High-Dimensional Application: ANN Training

7.1 Motivation

The benchmarks in Sections 3 to 5 have at most 100 variables and a known minimum. Practical problems usually have far more unknowns, so in this section the ADAM variants are used to train an artificial neural network (ANN). The network for the 100-dimensional problem, with layers $[100, 128, 64, 1]$, already has $P = \sum_l n_l(n_{l-1}+1) = 21\,249$ parameters. Their gradient is obtained with the adjoint method at about the cost of one forward pass, while central finite differences would need $2P = 42\,498$ loss evaluations. Since the loss of a network has no known minimizer, performance is measured by the prediction error on unseen data instead of the distance to w^*.

7.2 Network and Adjoint Gradient

For layers $l = 1, \dots, L$ with weights $\boldsymbol{W}^{(l)}$ and biases $\boldsymbol{b}^{(l)}$, and samples stored column-wise in $\boldsymbol{A}^{(0)} = \boldsymbol{X}$, the forward pass and the loss are

$$\boldsymbol{Z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{A}^{(l-1)} + \boldsymbol{b}^{(l)}, \qquad \boldsymbol{A}^{(l)} = \sigma\big(\boldsymbol{Z}^{(l)}\big), \qquad J = \frac{1}{n_L N_b}\big\|\boldsymbol{A}^{(L)} - \tilde{\boldsymbol{Y}}\big\|_F^2, \tag{29}$$

with $\sigma = \tanh$ in the hidden layers, a linear output layer and standardized targets $\tilde{\boldsymbol{Y}}$. The gradient is computed by reverse-mode differentiation, known as backpropagation [5] or the discrete adjoint method [1]. The adjoint $\boldsymbol{\Delta}^{(l)} = \partial J/\partial \boldsymbol{Z}^{(l)}$ is passed backwards from the output (Figure 12):

$$\boldsymbol{\Delta}^{(L)} = \frac{2}{n_L N_b}\big(\boldsymbol{A}^{(L)} - \tilde{\boldsymbol{Y}}\big), \qquad \boldsymbol{\Delta}^{(l-1)} = \big(\boldsymbol{W}^{(l)\mathsf{T}}\boldsymbol{\Delta}^{(l)}\big) \odot \sigma'\big(\boldsymbol{Z}^{(l-1)}\big), \tag{30}$$

$$\frac{\partial J}{\partial \boldsymbol{W}^{(l)}} = \boldsymbol{\Delta}^{(l)}\boldsymbol{A}^{(l-1)\mathsf{T}}, \qquad \frac{\partial J}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{\Delta}^{(l)}\mathbf{1}. \tag{31}$$

The gradients have the same structure as the parameters, so the optimizer step functions of the previous sections are used without changes.

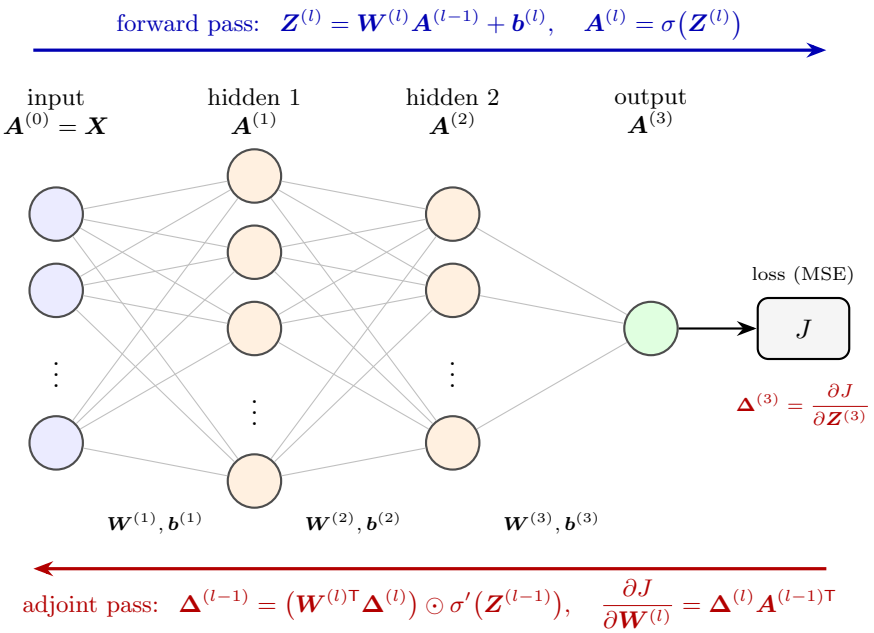


Figure 12: Fully connected network with forward pass (left to right) and adjoint pass (right to left).

Gradient check. The adjoint gradient was compared with central finite differences, $g_j^{\mathrm{FD}} = [J(\boldsymbol{\theta} + h\boldsymbol{e}_j) - J(\boldsymbol{\theta} - h\boldsymbol{e}_j)]/2h$ with $h = 10^{-6}$, for every weight and bias of a small network. The relative error $|g_j^{\mathrm{FD}} - g_j^{\mathrm{adj}}|/\max(1, |g_j^{\mathrm{FD}}|)$ stayed below 2.5×10^{-9} for all parameter arrays (Table 8).

Table 8: Gradient check: maximum relative error between adjoint and central finite-difference gradients (layers $[1, 5, 4, 1]$, tanh, $h = 10^{-6}$).

Parameter array	Size	Max. relative error e	Passed ($e < 10^{-5}$)
$\boldsymbol{W}^{(1)}$	5×1	1.27×10^{-9}	yes
$\boldsymbol{b}^{(1)}$	5×1	2.44×10^{-9}	yes
$\boldsymbol{W}^{(2)}$	4×5	1.94×10^{-9}	yes
$\boldsymbol{b}^{(2)}$	4×1	1.25×10^{-9}	yes
$\boldsymbol{W}^{(3)}$	1×4	3.07×10^{-10}	yes
$\boldsymbol{b}^{(3)}$	1×1	3.03×10^{-11}	yes

7.3 Training Setup

The data are split randomly into 70% training, 15% validation and 15% test samples. Inputs are scaled to $[-1, 1]$ and outputs standardized with training statistics, so a scaled MSE of 1 equals predicting the mean. The batch size B selects full-batch training ($B = N_{\mathrm{tr}}$), mini-batch training ($1 < B < N_{\mathrm{tr}}$) or SGD ($B = 1$); all results below use full batch. The weights with the lowest validation loss are kept and evaluated once on the test set (Algorithm 3).

ADAM, AdamW [2], AMSGrad [4] and Twin ADAM use the same training code; only the step function is exchanged. All runs use the same seed, so data, split and initial weights are identical, and the same hyperparameters (Table 9). The optimizers are compared by the test MSE (main criterion), the training and best validation MSE, the generalisation ratio $G = J_{\mathrm{te}}/J_{\mathrm{tr}}$ ($G \gg 1$ indicates overfitting) and the wall-clock time. The computational cost of E epochs is approximately

$$C \approx 6\, P\, k\, E\, N_{\mathrm{tr}}, \tag{32}$$

since a forward and an adjoint pass need about $2P$ and $4P$ operations per sample, with $k = 1$ for ADAM, AdamW and AMSGrad and $k = 2$ for Twin ADAM.

Table 9: Test problems and common settings.

Test problems	$\sin(0.2x)$ with noise (1D); Rosenbrock fit with $D = 2$ and 100 inputs; Concrete Compressive Strength data set (UCI)
Network (100D)	$[100, 128, 64, 1]$, $P = 21\,249$, $N = 100\,000$ samples in $[-2, 2]^{100}$
Activation	tanh (hidden), linear (output)
Hyperparameters	$\alpha = 0.01$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, AdamW $\lambda = 0.1$
Batch size	full batch ($B = N_{\mathrm{tr}}$)
Seed	1, reset before every run

Twin ADAM. The twin-gradient method of Section 6.4 [3] is applied to two independently initialised networks $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ trained on the same batches. Their ADAM steps $\boldsymbol{s}_1, \boldsymbol{s}_2$ are scaled by

$$\min_{\gamma_1, \gamma_2} \left\| (\boldsymbol{\theta}_1 + \gamma_1 \boldsymbol{s}_1) - (\boldsymbol{\theta}_2 + \gamma_2 \boldsymbol{s}_2) \right\|_2^2, \qquad \gamma_{1,2} \in [0.5,\, 3.5], \tag{33}$$

with $\gamma_1 = \gamma_2 = 1$ if the steps are nearly parallel. After each epoch the network with the lower validation loss is kept. Each update needs two gradient evaluations.

Algorithm 3 ANN training with an exchangeable ADAM-type optimizer (`ADAM_ANN.m`)

Require: Data $(\boldsymbol{X}, \boldsymbol{Y})$, layer widths, activation σ, epochs E, batch size B, step function `opt`, hyperparameters $(\alpha, \beta_1, \beta_2, \varepsilon)$

Split data 70/15/15; scale inputs to $[-1, 1]$, standardize outputs (training statistics)
Initialise $\boldsymbol{\theta}$; optimizer state $\leftarrow \emptyset$; $t \leftarrow 0$; $J_{\text{va}}^{\text{best}} \leftarrow \infty$
for $e = 1, \ldots, E$ **do**
    Shuffle the training samples
    for each batch $\mathcal{B}$ of size B **do**
        $\boldsymbol{A}^{(L)} \leftarrow$ forward pass on $\mathcal{B}$ ▷ store $\boldsymbol{Z}^{(l)}, \boldsymbol{A}^{(l)}$
        $\nabla_{\boldsymbol{\theta}} J \leftarrow$ adjoint pass on $\mathcal{B}$ ▷ Section 7.2
        $t \leftarrow t + 1$
        $[\boldsymbol{\theta}, \text{state}] \leftarrow \texttt{opt}(\boldsymbol{\theta}, \nabla_{\boldsymbol{\theta}} J, \text{state}, t, \text{hp})$ ▷ ADAM, AdamW, AMSGrad
    end for
    Evaluate J_{tr} and J_{va} on the complete sets
    if $J_{\text{va}} < J_{\text{va}}^{\text{best}}$ **then**
        $J_{\text{va}}^{\text{best}} \leftarrow J_{\text{va}}$; $\boldsymbol{\theta}^{\text{best}} \leftarrow \boldsymbol{\theta}$
    end if
end for
Evaluate J_{te} with $\boldsymbol{\theta}^{\text{best}}$
return $\boldsymbol{\theta}^{\text{best}}$, loss histories, J_{te}

7.4 Results

Table 10 summarises all ANN benchmarks.

Table 10: ANN benchmark results (scaled MSE, seed 1, settings of Table 9).

Optimizer	Train MSE	Best val. MSE	Test MSE	G	Time [s]
1D: $\sin(0.2x)$ *with noise*					
ADAM	4.67×10^{-3}	1.32×10^{-2}	1.20×10^{-2}	2.56	2.99
AdamW	1.64×10^{-1}	1.15×10^{-1}	1.18×10^{-1}	0.72	3.80
AMSGrad	2.34×10^{-1}	3.24×10^{-1}	3.19×10^{-1}	1.37	3.91
Twin ADAM	9.42×10^{-3}	1.36×10^{-2}	1.30×10^{-2}	1.38	6.99
Rosenbrock 2D					
ADAM	5.16×10^{-5}	1.48×10^{-4}	4.70×10^{-5}	0.91	1.99
AdamW	2.21×10^{-3}	5.04×10^{-3}	2.54×10^{-3}	1.14	1.98
AMSGrad	1.02×10^{-4}	3.07×10^{-4}	9.36×10^{-5}	0.92	1.95
Twin ADAM	8.31×10^{-5}	2.50×10^{-4}	8.39×10^{-5}	1.0	3.2
Rosenbrock 100D					
ADAM	2.01×10^{-1}	5.44×10^{-1}	5.44×10^{-1}	2.70	493.18
AdamW	2.95×10^{-1}	6.27×10^{-1}	6.44×10^{-1}	2.18	531.00
AMSGrad	1.94×10^{-1}	5.26×10^{-1}	5.23×10^{-1}	2.69	573.19
Twin ADAM	5.21×10^{-1}	7.12×10^{-1}	7.25×10^{-1}	1.39	950.16
Real-world data set: Concrete dataset					
ADAM	4.68×10^{-2}	9.23×10^{-2}	1.15×10^{-1}	2.46	3.37
AdamW	4.20×10^{-2}	7.61×10^{-2}	1.13×10^{-1}	2.71	3.34
AMSGrad	8.51×10^{-2}	1.05×10^{-1}	1.42×10^{-1}	1.67	3.56
Twin ADAM	4.83×10^{-2}	7.48×10^{-2}	1.53×10^{-1}	3.17	6.04

Low-dimensional fits. The 1D and 2D cases check the complete pipeline: the prediction follows $\sin(0.2x)$ (Figure 13) and reproduces the curved valley of the 2D Rosenbrock function

(Figure 14). ADAM gives the lowest test MSE in both cases. The test MSE of AdamW is about 10 times (1D) and 54 times (2D) higher, and that of AMSGrad about 27 times higher in 1D.

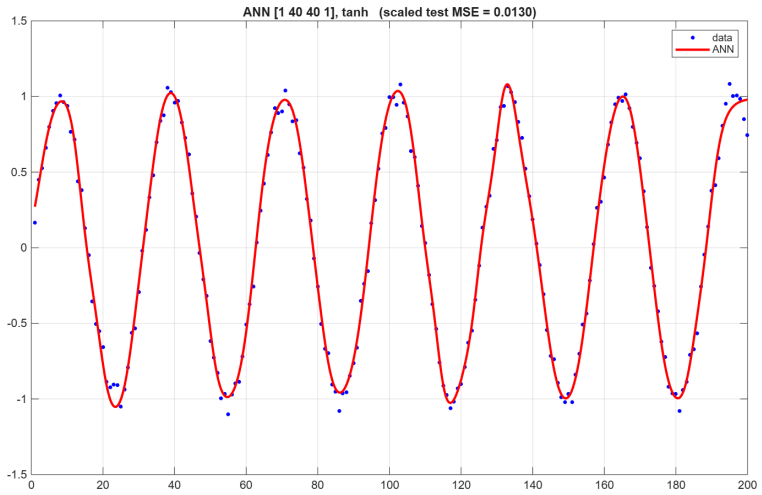
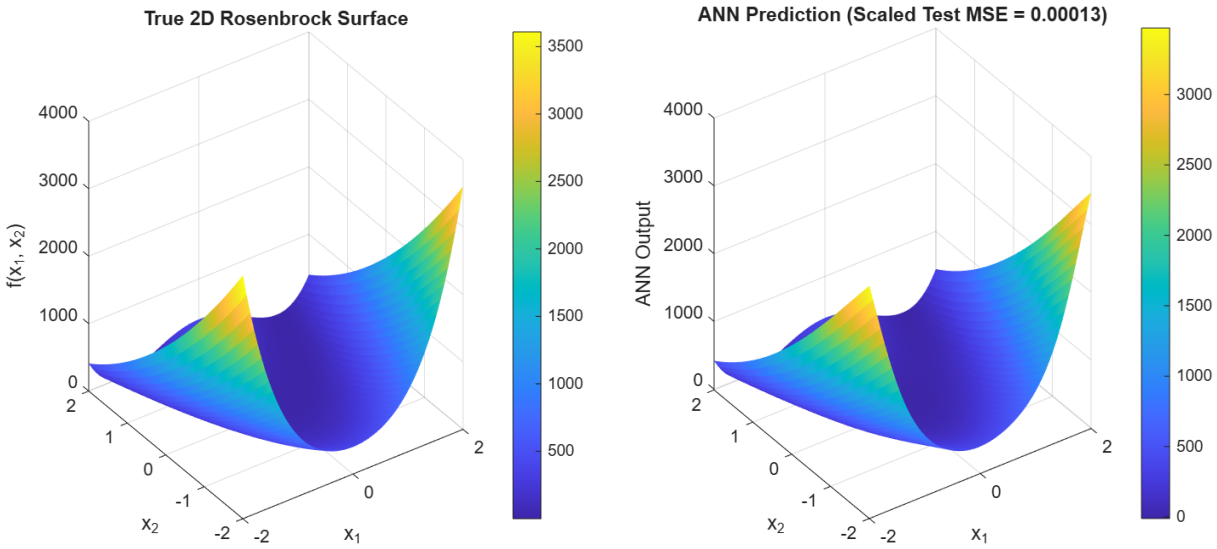


Figure 13: ANN approximation of $\sin(0.2x)$ with noise (ADAM, settings of Table 9).

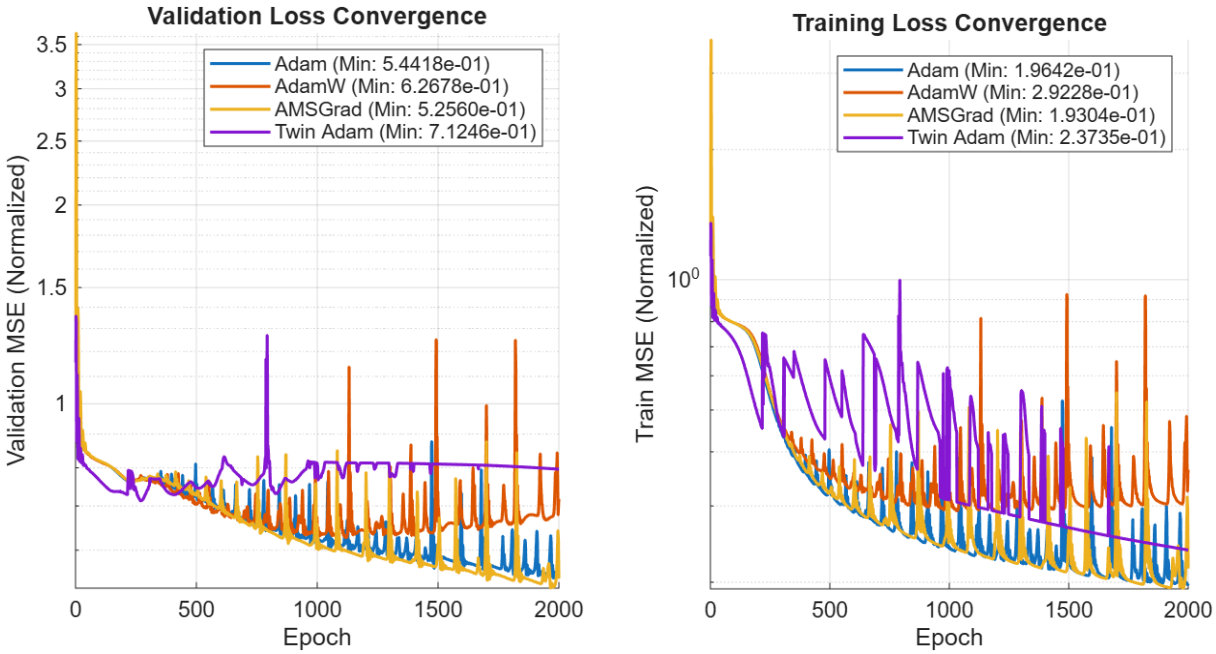


(a) Original Function **(b)** ANN prediction

Figure 14: ANN approximation of the 2D Rosenbrock function (ADAM, settings of Table 9).

100-dimensional Rosenbrock. Twin ADAM has the lowest validation loss between about epoch 100 and 250 but then stagnates at about 0.71 (Figure 15). ADAM, AdamW and AMSGrad show repeated spikes after about epoch 300, largest for AdamW. AMSGrad reaches the lowest test MSE (5.23×10^{-1}), followed by ADAM (5.44×10^{-1}). With G between 1.39 and 2.70, all optimizers overfit to some degree despite $N = 100\,000$ samples.

Concrete data set. The Concrete Compressive Strength data set [6] contains 1030 laboratory tests with 8 inputs (seven mixture components in kg/m^3 and the age in days) and the compressive strength in MPa as output. AdamW reaches the lowest test MSE. The validation loss of ADAM and AMSGrad rises again after about 6500 and 7000 epochs, which shows the onset of overfitting (Figure 16).



(a) Validation loss

(b) Training loss

Figure 15: Convergence of ADAM, AdamW, AMSGrad and Twin ADAM for the 100-dimensional Rosenbrock regression.

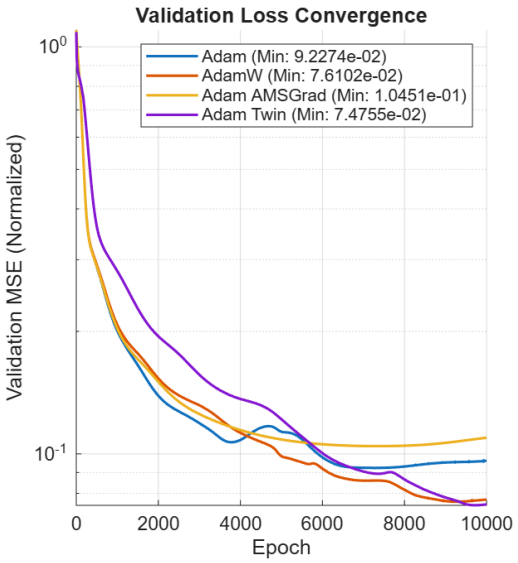


Figure 16: Validation loss of ADAM, AdamW, AMSGrad and Twin ADAM over the epochs for the Concrete Compressive Strength data set.

Twin ADAM. Twin ADAM did not reach a lower validation loss than ADAM on the 1D, 2D and 100D problems. On the Concrete data its validation loss was lower (7.48×10^{-2} against 9.23×10^{-2}), but its test MSE was the highest. Its time per epoch was 1.6 to 2.3 times that of ADAM (Figure 17).

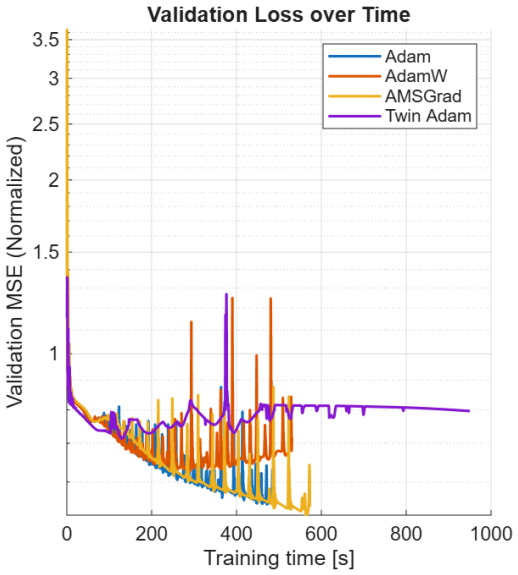


Figure 17: ADAM and Twin ADAM compared at equal computing time (100-dimensional Rosenbrock regression).

7.5 Discussion

The adjoint gradient agreed with finite differences to about 10^{-9} and made training with 21 249 unknowns practical. ADAM was best on the 1D and 2D fits and within about 4% of the best elsewhere, AMSGrad was best on the 100D problem and AdamW on the Concrete data. The ranking of the direct Rosenbrock problem (Section 5.4), where AdamW was best and AMSGrad worst, therefore does not carry over to network training. Twin ADAM doubled the cost per update without improving the networks, although it can help on the single-basin problems of Section 6.4. Since all runs use one seed and the same hyperparameters, repeated runs and a learning-rate study would show how robust this ranking is.

References

[1] Atılım Güneş Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.

[2] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization (AdamW). *arXiv preprint arXiv:1711.05101*, 2017.

[3] Anna De Magistris, Michiel E. Hochstenbach, and Gerardo Toraldo. A twin gradient method for unconstrained optimization. *arXiv preprint arXiv:2607.11575*, 2026.

[4] Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of Adam and beyond. In *International Conference on Learning Representations (ICLR)*, 2018.

[5] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[6] I-Cheng Yeh. Modeling of strength of high-performance concrete using artificial neural networks. *Cement and Concrete Research*, 28(12):1797–1808, 1998. Data set: UCI Machine Learning Repository, `https://doi.org/10.24432/C5PK67`.